C++ Cheatsheet

Last Updated : 11 Oct, 2025

This C++ cheat sheet is a quick reference guide for beginners and intermediates. It helps you learn or revise key C++ concepts without switching between multiple websites.

C++ is a high-level, general-purpose programming language created by Bjarne Stroustrup at Bell Labs in 1983. It is widely used for building software, games, operating systems, and performance-critical applications.

Basic Structure of C++ Program

```
// Header file for input and output
#include <iostream>

// To avoid using std:: before cout and cin
using namespace std;

// Main function - entry point of every C++ program
int main()
{
    // Display output on the screen
    cout << "Hello, World!" << endl;

    // Take input from the user
    int num;
    cout << "Enter a number: ";
    cin >> num;

    // Display the entered number
    cout << "You entered: " << num << endl;

    // Indicates successful program execution
    return 0;
}
```

Output

```
Hello, World!
Enter a number: You entered: 0
```

Comments

- Comments are used for providing an explanation of the code that makes it easier for others to understand the functionality of the code.
- They are not executed by the compiler.
- Comments can also be used to temporarily disable specific statements of code without deleting them.

There are two types of comments

1. **Single-line**: starts with //
2. **Multi-line:** starts with /* and ends with */.

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Single-line comment: This prints a greeting message
    cout << "Hello, World!" << endl;

    /*
       Multi-line comment:
       The next lines print some example numbers and messages.
       This shows how multi-line comments work in C++.
    */
    cout << "Number 1: 10" << endl;
    cout << "Number 2: 20" << endl;

    return 0; // End of the program
}
```

Output

```
Hello, World!
Number 1: 10
Number 2: 20
```

Variables

- A variable is a container used to store data values and must be declared before they can be used.
- You can declare multiple variables at the same time.
- A variable name can contain letters, digits, and underscores (_), and it must start with a letter or an underscore.

***Identifiers**: Unique names given to variables so they can be recognized individually in a program.*

***Constants**: Values that do not change during program execution.*

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Declaring variables
    int age = 20;
    double height = 5.7;
    char grade = 'A';
    string name = "John";

    // Printing variable values
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
    cout << "Height: " << height << endl;
    cout << "Grade: " << grade << endl;

    return 0;
}
```

Output

```
Name: John
Age: 20
Height: 5.7
Grade: A
```

Data Types

- Data types define the type of data a variable can store. Every variable in C++ must have a data type.
- C++ is a statically typed language, which means the data type of a variable is fixed at compile time and cannot change later.

Types of Data Types in C++

1. Integer (int)

- Stores whole numbers (e.g., 10, -5, 0). Can be used for counting, indexing, or arithmetic operations.
- Integers take 4 bytes of memory.

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Declaring an integer variable
    int age = 25;

    // Printing the value of the variable
    cout << "My age is: " << age << endl;

    return 0;
}
```

Output

```
My age is: 25
```

2. Floating Point (float)

- Stores decimal numbers with single precision (e.g., 3.14). Useful for measurements, fractions, and approximate calculations.
- It takes 4 bytes of memory.

```cpp
#include <iostream>
using namespace std;

int main()
{
    // Declaring a float variable
    float pi = 3.14;

    // Printing the value of the variable
```

```
    cout << "Value of pi: " << pi << endl;

    // Performing a simple arithmetic operation
    float radius = 2.0;
    float area = 3.14 * radius * radius;
    cout << "Area of circle with radius " << radius << " is: " << area << endl;

    return 0;
}
```

Output

```
Value of pi: 3.14
Area of circle with radius 2 is: 12.56
```

3. Double (double)

- Stores decimal numbers with double precision (more accurate than float, e.g., 3.14159). Used when more precision is required in calculations.
- It takes 8 bytes of memory.

```
#include <iostream>
using namespace std;

int main()
{
    // Declaring a double variable
    double price = 99.99;

    // Printing the value of the variable
    cout << "Price: " << price << endl;

    // Performing a simple calculation
    double discount = 10.5;
    double finalPrice = price - discount;
    cout << "Final Price after discount: " << finalPrice << endl;

    return 0;
}
```

Output

```
Price: 99.99
Final Price after discount: 89.49
```

4. Character (char)

- Stores a single character (e.g., 'A', 'b', '9'). Usually enclosed in single quotes and can be used in text processing.
- It takes 1 byte of memory.

```
#include <iostream>
using namespace std;
```

```
int main()
{
    // Declaring a char variable
    char grade = 'A';

    // Printing the value of the variable
    cout << "My grade is: " << grade << endl;

    return 0;
}
```

Output

```
My grade is: A
```

5. Boolean (bool)

- Stores only true or false values. Commonly used in conditions and logical operations.
- It takes 1 byte of memory.

```
#include <iostream>
using namespace std;

int main()
{
    // Declaring a bool variable
    bool isStudent = true;

    // Printing the value of the variable
    cout << "Is the person a student? " << isStudent << endl;

    // Changing the value
    isStudent = false;
    cout << "Is the person a student now? " << isStudent << endl;

    return 0;
}
```

6. String (string)

- Stores a sequence of characters (e.g., "Hello"). Used for text, names, messages, or sentences.
- We have to include <string> header file for using string class.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // Declaring a string variable
    string name = "Alice";

    // Printing the string
    cout << "Name: " << name << endl;
```

```
    // Concatenating strings
    string greeting = "Hello, " + name + "!";
    cout << greeting << endl;

    return 0;
}
```

Output

```
Name: Alice
Hello, Alice!
```

Input and Output

Input and Output in C++ (cin and cout) are part of the iostream library, used to read data from the user (cin) and display data on the screen (cout).

1. Input (cin)

- Used to take input from the user via the keyboard.
- Uses the extraction operator >> to read data into a variable.

2. Output (cout)

- Used to display information on the screen.
- Uses the insertion operator << to send data to the output.

```
#include <iostream>
using namespace std;

int main()
{
    // Declare variables
    string name;
    int age;

    // Taking input from the user
    cout << "Enter your name: ";
    cin >> name;
    cout << "Enter your age: ";
    cin >> age;

    // Displaying the input back to the user
    cout << "Hello, " << name << "! You are " << age << " years old." << endl;

    return 0;
}
```

Output:

```
Enter your name: Alice
Enter your age: 20
Hello, Alice! You are 20 years old.
```

Conditional Statements

- Conditional statements allow us to control the flow of the program based on certain conditions.
- It helps us to run a specific section of code based on a condition.

Types of conditional statements

1. if statement

- Executes a block of code only if the condition is true.
- The condition is usually a boolean expression. If the condition is false, the code inside the if block is skipped.

```
#include <iostream>
using namespace std;

int main()
{
    int i = 10;

    // If statement
    if (i < 15)
    {
        cout << "10 is less than 15";
    }

    return 0;
}
```

Output

```
10 is less than 15
```

2. if-else statement

- Executes one block if the condition is true and another block if it is false.
- Allows two possible paths of execution.

```
#include <iostream>
using namespace std;

int main()
{
    int i = 10;

    // If statement
    if (i < 15)
    {
        cout << "10 is less than 15";
    }

    // Else statement with the above if
    else
    {
        cout << "10 is not less than 15";
    }
```

```
    return 0;
}
```

Output

```
10 is less than 15
```

3. else if ladder

- else if Checks multiple conditions sequentially.
- Executes the block of the first true condition and skips the rest.
- Useful when there are more than two possible outcomes.

```c
#include <stdio.h>

int main()
{
    int marks = 85;

    // Assign grade based on marks
    if (marks >= 90)
    {
        printf("A\n");
    }
    else if (marks >= 80)
    {
        printf("B\n");
    }
    else if (marks >= 70)
    {
        printf("C\n");
    }
    else if (marks >= 60)
    {
        printf("D\n");
    }
    else
    {
        printf("F\n");
    }

    return 0;
}
```

Output

```
B
```

4. switch statement

- Executes one block out of many based on a variable's value.
- Each case must have a break to stop execution after a match.
- Useful when there are fixed known options.

```cpp
#include <iostream>
using namespace std;

int main()
{

    // Variable to the used as switch expression
    char x = 'A';

    // Switch statement with three cases
    switch (x)
    {
    case 'A':
        cout << "A";
        break;
    case 'B':
        cout << "B";
        break;
    default:
        cout << "Other than A and B";
        break;
    }
    return 0;
}
```

Output

```
A
```

5. Shorthand if else (Ternary Operator)

- Ternary operator (?:) is a short-hand for simple if-else statements.
- It evaluates a condition and returns one value if true and another if false.

```cpp
#include <iostream>
using namespace std;

int main()
{
    int x = 10, y = 20;

    // Using ternary operator
    int max_val = (x > y) ? x : y;

    cout << "The maximum value is " << max_val;
    return 0;
}
```

Output

```
The maximum value is 20
```

Loops

- Loopsare used to repeat a block of code multiple times until a condition is met.

- They save time and effort by avoiding writing the same code again and again.

Types of Loops

1. For Loop

- for loop repeats a block of code a fixed number of times.
- Usually has initialization, condition, and increment/decrement in one line.
- Best used when the number of iterations is known.

```
#include <bits/stdc++.h>
using namespace std;
```



```
    // for loop to print "Hi" 5 times
    for (int i = 5; i < 10; i++)
    {
        cout << "Hi" << endl;
    }

    return 0;
}
```

Output

```
Hi
Hi
Hi
Hi
Hi
```

2. While Loop

- It repeats a block of code as long as the condition is true.
- The condition is checked before executing the code.
- Best used when the number of iterations is not known in advance.

```
#include <iostream>
using namespace std;

int main()
{

    // while loop to print numbers from 1 to 5
    int i = 0;
    while (i < 5)
    {
        cout << "Hi" << endl;
        i++;
    }

    return 0;
```

```
}
```

Output

```
Hi
Hi
Hi
Hi
Hi
```

3. Do-While Loop

- It is similar to while loop, but executes the code at least once.
- The condition is checked after executing the block.
- Useful when you want the code to run once before checking the condition.

```cpp
#include <iostream>
using namespace std;

int main()
{

    // do-while loop to print "Hi" 5 times
    int i = 0;
    do
    {
        cout << "Hi" << endl;
        i++;
    } while (i < 5);

    return 0;
}
```

Output

```
Hi
Hi
Hi
Hi
Hi
```

Arrays

- An array is a collection of multiple values of the same type stored under a single name.
- Each value in the array can be accessed using an index (starting from 0).
- Arrays are useful to store and manage a list of similar items efficiently.

```cpp
#include <iostream>
using namespace std;

int main()
{
    // declaring and initializing an array of size 5
    int arr[5] = {2, 4, 8, 12, 16};

    // printing array elements
```

```
    for (int i = 0; i < 5; i++)
    {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output

```
2 4 8 12 16
```

Multi-Dimensional Arrays

- A multi-dimensional array is an array of arrays.
- The most common type is a 2D array, which can be thought of as a table or matrix with rows and columns.
- Values are accessed using multiple indices, e.g., array[row][column].

```
#include <iostream>
using namespace std;

int main()
{

    // declaring and initializing a 2D array
    // with 3 rows and 4 columns
    int matrix[3][4] = {{1, 2, 3, 4}, {5, 6, 7, 8}, {9, 10, 11, 12}};

    // printing the elements of the 2D array
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            cout << matrix[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Output

```
1 2 3 4
5 6 7 8
9 10 11 12
```

Vectors

- A vector is a dynamic array that can grow or shrink in size automatically.
- It is part of the C++ Standard Template Library (STL), so you need #include <vector>.
- Elements in a vector are accessed using indices just like arrays.
- Vectors provide useful functions like push_back(), pop_back(), size(), and clear().

```
#include <iostream>
#include <vector>
```

```
using namespace std;
int main()
{

    // Declares an empty vector
    vector<int> v1;

    // Declares vector with given size
    // and fills it with a value
    vector<int> v2(3, 5);

    // Print items of v2
    for (int x : v2)
    {
        cout << x << " ";
    }

    cout << endl;

    // Initializes vector using initializer list.
    vector<int> v3 = {1, 2, 3};

    // Print items of v3
    for (int x : v3)
    {
        cout << x << " ";
    }

    return 0;
}
```

Output

```
5 5 5
1 2 3
```

References

- A reference is an alias for another variable, meaning it refers to the same memory location.
- Once a reference is initialized, it cannot be changed to refer to another variable.

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;

    // ref is a reference to x.
    int &ref = x;

    // printing value using ref
    cout << ref << endl;

    // Changing the value and printing again
    ref = 22;
    cout << ref;

    return 0;
```

```
}
```

Output

```
10
22
```

Pointers

- A pointer is a variable that stores the memory address of another variable.
- Pointers can be reassigned to point to different variables.

```
#include <iostream>
using namespace std;

int main()
{
    int var = 10;

    // declare pointer and store address of x
    int *ptr = &var;

    // print value and address
    cout << "Value of x: " << var << endl;
    cout << "Address of x: " << &var << endl;
    cout << "Value stored in pointer ptr: " << ptr << endl;
    cout << "Value pointed to by ptr: " << *ptr << endl;

    return 0;
}
```

Output

```
Value of x: 10
Address of x: 0x7ffed805557c
Value stored in pointer ptr: 0x7ffed805557c
Value pointed to by ptr: 10
```

Functions

- A function is a block of code designed to perform a specific task.
- It help avoid repeating code and make programs organized and readable.
- A function can take inputs (parameters) and return a value, but both are optional.
- Functions are called/invoked whenever you need to execute that task.

```
#include <iostream>
using namespace std;

// Function declaration and definition
void greet()
{
    cout << "Hello, World!" << endl;
}

int main()
{
```

```
    // Calling the function
    greet();

    return 0;
}
```

Output

```
Hello, World!
```

String Functions

There are several string functions present in Standard Template Library in C++ that are used to perform operations on strings. Some of the commonly used string functions are:

1. length()

- Returns the length of a string.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    // Declare a string
    string name = "GeeksforGeeks";

    // Find and print the length of the string
    cout << "The length of the name is: " << name.length() << endl;

    return 0;
}
```

Output

```
The length of the name is: 13
```

2. substr()

- It is used to extract a substring from a given string.

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str = "GeeksforGeeks";

    // Extracts a substring starting from
    // index 1 with a length of 5
    string sub = str.substr(1, 5);

    cout << "Substring: " << sub << endl;
```

```
    return 0;
}
```

Output

```
Substring: eeksf
```

3. append()

- Appends a string at the end of the given string.

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string str = "Geeksfor";

    str.append("Geeks");

    cout << "Appended string: " << str << endl;

    return 0;
}
```

Output

```
Appended string: GeeksforGeeks
```

4. compare()

- It is used to compare two strings lexicographically.

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str1 = "Geeks";
    string str2 = "for";
    string str3 = "Geeks";

    int result1 = str1.compare(str2);
    cout << "Comparison result: " << result1 << endl;

    int result2 = str1.compare(str3);
    cout << "Comparison result: " << result2 << endl;

    return 0;
}
```

Output

```
Comparison result: -31
Comparison result: 0
```

5. empty()

- It is used to check if a string is empty.

```
#include <iostream>
using namespace std;

int main()
{
    string str1 = "GeeksforGeeks";
    string str2 = "";

    if (str1.empty())
        cout << "str1 is empty" << endl;
    else
        cout << "str1 is not empty" << endl;

    if (str2.empty())
        cout << "str2 is empty" << endl;
    else
        cout << "str2 is not empty" << endl;
    return 0;
}
```

Output

```
str1 is not empty
str2 is empty
```

Math Functions

Function	Description	Example
min(x, y)	Returns the minimum value of x and y.	cout << min(10, 20);
max(x, y)	Returns the maximum value of x and y.	cout << max(10, 20);
sqrt(x)	Returns the square root of x.	cout << sqrt(25);
ceil(x)	It rounds up the value x to its nearest integer.	double ceilX = ceil(3.14159);
floor(x)	It rounds the value of x downwards to the nearest integer.	double floorX = floor(3.14159);

Function	Description	Example
pow(x,n)	It returns the value x raised to the power of y	double result = pow(3.0, 2.0);

Object-Oriented Programming

Object-oriented programming generally means storing data in the form of classes and objects.

Class and Objects

- Class: A class is a user-defined data type that contains its data members and member functions. A class is a blueprint for objects having similar attributes and behavior.
- **Objects:** An object is an instance or a variable of the class.

Pillars of OOPs

1. Encapsulation

- Encapsulation means bundling data (variables) and functions (methods) together in a class.
- It hides the internal details of an object from the outside world (data hiding).
- Access to data is provided through public functions (getters/setters).
- Helps in preventing accidental modification of important data.

2. Abstraction

- Abstraction is hiding complex implementation details and showing only the necessary features.
- It helps in simplifying the program for the user.
- Achieved in C++ using abstract classes and pure virtual functions.
- Example: A car driver knows how to drive, but doesn't need to know the engine details.

3. Inheritance

- Inheritance allows a class (child/derived class) to acquire properties and behavior of another class (parent/base class).
- Promotes code reusability.
- Types in C++: single, multiple, multilevel, hierarchical, hybrid.
- Example: A Car class can inherit from a Vehicle class to reuse common features.

4. Polymorphism

- Polymorphism means one entity can take many forms.
- Two types: Compile-time (function overloading, operator overloading) , Run-time (virtual functions)
- Helps in using the same interface for different types of objects.
- Example: A print() function can behave differently for int, float, or string.

File Handling

- **File handling** means reading data from a file and manipulating the data of a file.

File Handling Operations

- 1. Open a file: We can use open() member function of ofstream class to open a file.
- 2. Read a file: We can use getline() member function of ifstream class to read a file.
- 3. Write to a file: We can use << operator to write to a file after opening a file with the object of ofstream class.

```
#include <fstream>
#include <iostream>
#include <string>

using namespace std;

int main()
{
    ofstream outputFile("example.txt");

    // Open the file for writing
    outputFile.open("example.txt");
    if (outputFile.is_open()) {

        // Write data to the file
        outputFile << "Hello, World!" << endl;
        outputFile << 42 << endl;
        outputFile.close(); // Close the file
    }
    else {

        // Failed to open the file
        cout << "Error opening the file for writing."
             << endl;
        return 1;
    }

    // Reading from a file
    ifstream inputFile("example.txt");
    if (inputFile.is_open()) {
        string line;
        while (getline(inputFile, line)) {
            // Print each line
            cout << line << endl;
        }
        // Close the file
        inputFile.close();
    }
    else {

        // Failed to open the file
        cout << "Error opening the file for reading."
             << endl;
        return 1;
    }

    return 0;
}
```

This C++ cheat sheet can serve as a reference guide for programmers that provides quick access to concepts of C++.

Comment

A abhish... + Follow 53

Article Tags: C++ GFG Sheets

Explore

C++ Basics

Core Concepts

OOP in C++

Standard Template Library(STL)

Practice & Problems

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved